

SLAM2REF: Advancing Long-Term Mapping with 3D LiDAR and Reference Map Integration for Precise 6-DoF Trajectory Estimation and Map Extension

Miguel A. Vega-Torres^{1*}, Alexander Braun¹ and André Borrmann¹

^{1*}Chair of Computational Modeling and Simulation, Technical
University of Munich, Arcisstraße 21, Munich, 80333, Bavaria, Germany.

*Corresponding author(s). E-mail(s): miguel.vega@tum.de;

Abstract

This paper presents a pioneering solution to the task of integrating mobile 3D LiDAR and inertial measurement unit (IMU) data with existing building information models or point clouds, which is crucial for achieving precise long-term localization and mapping in indoor, GPS-denied environments. Our proposed framework, SLAM2REF, introduces a novel approach for automatic alignment and map extension utilizing reference 3D maps. The methodology is supported by a sophisticated multi-session anchoring technique, which integrates novel descriptors and registration methodologies. Real-world experiments reveal the framework's remarkable robustness and accuracy, surpassing current state-of-the-art methods. Our open-source framework's significance lies in its contribution to resilient map data management, enhancing processes across diverse sectors such as construction site monitoring, emergency response, disaster management, and others, where fast-updated digital 3D maps contribute to better decision-making and productivity. Moreover, it offers advancements in localization and mapping research. Link to the repository: <https://github.com/MigVega/SLAM2REF>, Data: <https://doi.org/10.14459/2024mp1743877>.

Keywords: LiDAR, Multi-Session SLAM, Pose-Graph Optimization, Loop Closure, Long-term Mapping, Change Detection, BIM Update, 3D Indoor Localization and Mapping

1 Introduction

Nowadays, mobile mapping systems incorporated into mobile robots or handheld devices equipped with sensors and applying state-of-the-art Simultaneous Localization and Mapping (SLAM) algorithms allow the quick creation of updated 3D maps. However, these maps are in their local coordinates systems and, therefore, separated from any prior information. Additionally, they might contain potential drift issues, rendering them unsuitable for comparative analysis or change detection.

Several real-world applications require the capacity to align, compare, and manage 3D data received at various intervals that may be separated by lengthy intervals of time. This process is referred to as long-term map management.

Long-term map management is crucial since the real world constantly evolves and changes. This applies to humans who want to utilize the map to comprehend the current situation and its evolution and to autonomous robots for effective and fast navigation.

Moreover, achieving accurate alignment and effective management of extensive datasets represent significant challenges in enabling the creation of Digital Twins (DTs) for cities and buildings (Borrmann et al., 2024; Mylonas et al., 2021). As explained by (Botín-Sanabria et al., 2022), in complex implementations, automatic alignment of 3D data becomes imperative to achieve DTs with maturity levels of 3 or higher. Such levels necessitate the augmentation of models with a continuous flow of real-world information.

An automatic map alignment and change detection pipeline also contribute to the seamless integration of mapping devices into existing workflows in the industry. A recent survey revealed that the compatibility of mapping devices with existing tools is, after the budget, the second most crucial barrier surrounding the usage of mobile mapping devices (NavVis et al., 2022).

For example, an up-to-date 3D digital map can help construction site managers promptly distinguish as-planned and as-built differences, thus reducing the probability of long-schedule delays and high-cost overruns. Similarly, an updated 3D representation of the site can also help first responders during an emergency to improve situational awareness and enable decision-making to save lives effectively and safely (Alliez et al., 2020; He et al., 2021).

Furthermore, if a robot can align the measurements of an onboard sensor with a reference map (i.e., the robot can localize itself within the map), the semantically enriched building information model (BIM model) or the reference map can serve as a valuable source of information for various autonomous robotic activities. These activities include but are not limited to path planning (Dugstad et al., 2022), object inspection (Kim and Peavy, 2022), and maintenance and repair operations (Kim et al., 2021).

Global Positioning System (GPS) can be a viable option for outdoor localization and rough alignment. However, for indoor environments, GPS is often impractical because it requires a direct line of sight to at least four satellites—three to determine the 3D position and one for time correction. To address this, various Indoor Positioning System (IPS) alternatives use radio signals, such as Wi-Fi or Bluetooth, as well as

AprilTags (de Teruel et al., 2017; Kayhani et al., 2022, 2023; Koide et al., 2022). The downside of these systems is that they require additional strategically placed sensors or landmarks, which can increase the cost and effort of implementing such a positioning system. Nevertheless, although not always accessible, 3D prior maps of buildings are increasingly becoming standard in modern construction. These maps, often in the form of BIM models or point clouds, document the state of the building during and after construction or in the design phases.

Besides being useful for autonomous robotic tasks, aligning sensor data with an accurate reference map allows the retrieval of the sensor’s precise 6 Degrees of Freedom (DoF) ground truth (GT) poses in the entire trajectory.

These GT poses serve multiple functions. They enable precise identification of the capture locations of point clouds and images necessary for generating an accurate, updated 3D map. Additionally, they facilitate the assessment of the efficacy of SLAM, odometry, and localization algorithms. This capability is particularly crucial for advancing research and development in this field.

Historically, obtaining GT poses has necessitated costly equipment like real-time kinematic (RTK)-corrected global navigation satellite system (GNSS) for outdoor environments or laser trackers and motion capture systems for indoor settings (Liu et al., 2021). However, the expensive costs associated with these methods pose a substantial barrier for individual researchers. Additionally, acquiring dense GT poses for extended trajectories, especially in indoor scenarios, has been found to be challenging (Zhang et al., 2023).

Recent studies, such as by the ConSLAM (Trzeciak et al., 2023b,a) and Newer College (Ramezani et al., 2020; Zhang et al., 2022) datasets, have leveraged Terrestrial Laser Scanner (TLS) point clouds—providing millimeter-precise 3D scans of the environment—to be used as reference GT map and overcome these limitations. Through semi-automatic techniques, researchers have effectively aligned mobile Light Detection and Ranging (LiDAR) measurements with TLS point clouds. This advancement represents a significant step forward in SLAM research towards automatic, accurate GT pose acquisition methods suitable for both large indoor and outdoor scenarios.

To enable long-term map management and the automatic retrieval of precise 6-DoF poses for mobile LiDAR-based localization and mapping research, this study proposes **SLAM2REF**, an open-source¹ framework that uses a BIM model or a pre-existing point cloud as a reference map to allow an automatic alignment and correction of a map created with SLAM or odometry systems.

Herein, we adopt the term *reference map* to encompass a spectrum of environmental representations, such as designated BIM models, point clouds, or meshes.

As will be discussed in Section 3, several researchers have investigated using a reference map for robot localization (Vega Torres et al., 2022). However, only a few aim to create an accurate, updated 3D map that is aligned and corrected with the information in the reference map.

Furthermore, most research methods demand a reasonably good estimate of the robot’s initial position, which must also be within the reference map. In addition,

¹Link to the repository: <https://github.com/MigVega/SLAM2REF>

nearly no approach takes discrepancies between the reference map and the current state of the real-world (Scan-Map deviations) into account.

Scan-Map deviations can be classified into three categories: Firstly, the deviations coming from the presence of clutter or furniture absent in the reference map; Secondly, deviations due to the presence of dynamic (i.e., moving) elements in the environment while scanning; and thirdly, the presence of alterations on the permanent elements of the building, such as walls and columns. In this research, we focus on addressing the first two categories of deviations. Nonetheless, minor discrepancies in some permanent elements of the environment, such as holes or slight shifts in single columns or walls, would not hinder the successful implementation of our framework.

In general, while we allow Scan-Map deviations, we presume that the reference map remains a reliable map suited for localization, i.e., the BIM model or reference point cloud has enough features that comply geometrically with the current state of the environment.

To address the previously described research gaps, we present SLAM2REF, a novel framework that integrates 3D LiDAR data and Inertial Measurement Units (IMU) measurements with a reference map to achieve precise pose estimation, enabling also map extension and long-term map management.

The three key components and functionalities of our framework are the following:

- An automatic method that enables the creation of accurate Occupancy Grid Map (OGM)s and 3D session data from large-scale building information models (BIM models) or point clouds.
- A pipeline that leverages fast place recognition and multi-session anchoring to allow the alignment and correction of drifted session acquired with SLAM or LiDAR-inertial odometry systems. Provided that the reference map is accurate enough, the framework allows the retrieval of the 6 DoF poses of the entire trajectory, also enabling map extension, and surpassing state-of-the-art methods such as the one introduced by (Trzeciak et al., 2023a).
- A module that allows the analysis of the acquired aligned data, providing not only positive but also negative difference detection for an updated 3D map visualization.

We demonstrate the effectiveness of SLAM2REF through extensive experiments in various large-scale indoor GPS-denied real-world scenarios, showcasing its ability to achieve centimeter-level accuracy in trajectory estimation and robust map alignment over extended periods. Additionally, we demonstrate that the method enables the robust automatic alignment of the data with a reference BIM model, which does not contain clutter, furniture, or dynamic elements as the real-world data.

This is achieved through innovative feature descriptors based on the widely used Scan Context descriptor (Kim et al., 2021) and a novel YawGICP registration algorithm built based on the Open3D Generalized Iterative Closest Point (ICP) (GICP) method. Additionally, we incorporate motion distortion correction of individual scans by integrating IMU measurements to create continuous-time trajectories inspired by the Direct LiDAR Inertial Odometry system (Chen et al., 2023). These elements are holistically integrated into a multi-session anchoring framework that enables the registration of drifted SLAM session data with a reference map.

While our framework draws significant inspiration from LT-SLAM [Kim and Kim \(2022\)](#), our method is able to retrieve ground truth poses when an accurate reference map is available. Furthermore, our method incorporates motion distortion correction and is well-suited for indoor scenarios. It also can utilize any 3D map, such as point clouds or BIM models, as a reference, thus not being limited to the registration of session data pairs.

The following is the structure of the remainder of this paper.

Section 2 introduces the factor graph problem formulation of SLAM as well as of multi-session anchoring to align different sessions in a unified coordinate system.

Section 3 covers work on map-based LiDAR localization and mapping.

Section 4 introduces our modular SLAM2REF framework, divided into three main steps: **Step 1.** Generation of Session Data (SD) from a reference map, **Step 2.** Introduces the reference map-based multi-session anchoring method, which allows the alignment and correction of new session data with the reference map and **Step 3.** Change detection and meshing of new or removed elements in the environment.

Section 5 explains the experimental parameters and implementation details, followed by the results and analysis in section 6.

Sections 7 and 8, present the discussion and limitations related to the proposed pipeline and results.

Finally, sections 9 and 10 summarize what we have accomplished and bring our work to a close by discussing possible future research directions.

2 Theoretical background

Before presenting the current state-of-the-art methodologies, an introduction to the theoretical concepts behind localization and mapping algorithms, as well as the multi-session anchoring process employed in this research, is presented. For better understanding, a table with all mathematical variables and the corresponding description can be found in the appendix A.

In multi-session anchoring, similar to SLAM or a tracking scenario, the objective is to optimize the posterior probability of the poses in a trajectory based on collected measurements. In other words, we aim to find the poses for which the provided measurements have the highest probability.

However, in multi-session anchoring, we also aim to find the best alignment between sessions. Each session consists of successive sensor data collected from a specific location at varying time intervals.

These types of problems can be formulated as a Maximum A Posteriori (MAP) estimate that maximizes the posterior density $p(X|z)$ of the states X given the measurements Z . Instead of using Bayes Net, the problem can be considered as a factor graph factorization in which each factor is proportional to a conditional probability density.

While Bayesian nets provide a practical modeling framework, factor graphs facilitate rapid inference. Like Bayesian networks, factor graphs enable the representation of a joint density as a product of factors ([Dellaert et al., 2017](#)).

In robotics, various challenges, including pose estimation, planning, and optimal control, often involve solving optimization problems. These problems typically center around maximizing or minimizing objectives composed of numerous local factors or terms specific to small subsets of variables. Factor graphs allow the encapsulation of this local structure, with factors representing functions related to subsets of variables (Dellaert, 2021).

A factor graph $F = (\mathcal{U}, \mathcal{V}, \mathcal{E})$ comprises nodes connected by edges $e_{ij} \in \mathcal{E}$. The nodes can be of two types: factors $\phi_i \in \mathcal{U}$ and variables $x_i \in \mathcal{V}$. The factor graph represents the factorization of a global function, where each factor is a function of the variables in its adjacency set. Given that X_i is the group of variables x_i connected to a factor ϕ_i , a factor graph specifies the factorization of a global function $\phi(X)$ as

$$\phi(X) = \prod_i \phi_i(X_i).$$

Stated differently, each factor ϕ_i relies solely on the adjacent variables X_i and is connected to other factors via the edges e_{ij} .

An elegant representation of a SLAM problem is called *pose SLAM*, which eliminates the need to directly include landmarks in the optimization process. The focus of pose SLAM is to predict the robot's trajectory based on constraints from odometry and loop closures between the different poses in a trajectory (Jurić et al., 2021). These odometry constraints, describing the relative poses, can be derived from various sources (e.g., camera or wheel encoders); in this case, we use IMU and LiDAR measurements, as it will be described later in 4.2.1.

In general, MAP inference involves maximizing the product of all factor graph potentials for any arbitrary factor graph (Dellaert et al., 2017).

$$X^{\text{MAP}} = \underset{x}{\operatorname{argmax}} \prod_i \phi_i(X_i).$$

Assuming that all factors can be modeled by a measurement function h_i , with normally distributed priors and factors from measurements z_i with zero-mean Gaussian noise models Σ_i , then we have the following conditional density $p(z_i|x_i, l_i)$ on the measurement z_i .

$$p(z_i|x_i, l_i) = \mathcal{N}(z_i; h_i(x_i, l_i), \Sigma_i) = \frac{1}{\sqrt{|2\pi\Sigma_i|}} \exp \left\{ -\frac{1}{2} \|h_i(x_i, l_i) - z_i\|_{\Sigma_i}^2 \right\}.$$

Thus, we face factors that are proportional to:

$$\phi_i(X_i) \propto \exp \left\{ -\frac{1}{2} \|h_i(X_i) - z_i\|_{\Sigma_i}^2 \right\},$$

Taking the negative log of Eq. (2) and dropping the factor 1/2 allows us to instead minimize a sum of non-linear least squares:

$$\begin{aligned}
X^{\text{MAP}} &= \underset{x}{\operatorname{argmin}} -\log \prod_i \phi_i(X_i). \\
&= \underset{x}{\operatorname{argmin}} \sum_i \|h_i(X_i) - z_i\|_{\Sigma_i}^2.
\end{aligned} \tag{1}$$

In the context of multi-session anchoring, inter-session, or between sessions, loop closure detections, also called *encounters* $\mathbf{c}$ (which are also poses in the special Euclidean group $\text{SE}(3)$), can be added to the non-linear least squares formulation in Eq. (1) with the following Gaussian measurement equation:

$$\mathbf{c} = h(\mathbf{x}_R, \mathbf{x}_Q) + \eta,$$

where $h(\cdot)$ is a relative measurement prediction function, and η is a normally distributed zero-mean measurement noise with covariance Σ_c . Furthermore, $\mathbf{x}_R$ and $\mathbf{x}_Q$ are the sensor poses in the two sessions $\mathcal{S}_R$ and $\mathcal{S}_Q$, respectively. This yields the following conditional density $p(\mathbf{c}|\mathbf{x}_R, \mathbf{x}_Q)$ on the measurement $\mathbf{c}$

$$p(\mathbf{c}|\mathbf{x}_R, \mathbf{x}_Q) = \frac{1}{\sqrt{|2\pi\Sigma_c|}} \exp \left\{ -\frac{1}{2} \|h(\mathbf{x}_R, \mathbf{x}_Q) - \mathbf{c}\|_{\Sigma_c}^2 \right\}.$$

Similarly, an odometry model $f(\cdot)$, which usually incorporates a scan-matching process, among other techniques, produces constraints $\mathbf{u}_i^s$ between consecutive poses: $\mathbf{x}_i$ and $\mathbf{x}_{i+1}$.

Unifying the encounter measurement model $h(\cdot)$ together with the odometry model $f(\cdot)$ in Eq. (1), we obtain the following equation (omitting intra-session loop closures for simplicity).

$$\begin{aligned}
X^{\text{MAP}} &= \underset{x}{\operatorname{argmin}} \left\{ \sum_{\mathcal{S}} \left(\|\mathbf{p}_s - \mathbf{x}_{s,0}\|_{\Sigma_p}^2 + \sum_{i \in M_s} \|f_i(\mathbf{x}_{s,i}, \mathbf{u}_{s,i}) - \mathbf{x}_{s,i+1}\|_{\Sigma_o}^2 \right) \right. \\
&\quad \left. + \sum_{j \in N_e} \|h_j(\mathbf{x}_{R,j}, \mathbf{x}_{Q,j}) - \mathbf{c}_j\|_{\Sigma_c}^2 \right\}
\end{aligned} \tag{2}$$

Where $\mathcal{S} \in \{\mathcal{S}_Q, \mathcal{S}_R\}$, M_s is the number of poses in the session $\mathcal{S}$, and N_e is the number of encounters between sessions.

Here, we directly incorporate the initial pose of each session as a prior factor $\mathbf{p}_s$. This fixes the initial pose to the origin, effectively eliminating that gauge of freedom, i.e., assigning a local reference coordinate system to each session.

As in a multi-robot mapping problem, having two sessions or more requires a strategy to handle the fact that the sessions can have different initial poses and, therefore, other initialization prior (Lajoie and Beltrame, 2024).

We employ anchor nodes to address this problem and facilitate the integration of inter-session constraints.

The anchor Δ_Q is a $\text{SE}(3)$ pose for the session $\mathcal{S}_Q$ that determines how the entire trajectory is positioned concerning a global coordinate frame.

Essentially, we maintain the individual pose graphs of each session in their respective local frames and bind them with anchor factors to the global frame. For each session, an anchor node is added to the pose graph problem as the first pose of the session; this pose can be selected arbitrarily (usually set to the origin).

During the initial encounter, no modifications are made to the pose graphs of the respective sessions; only the anchor nodes change, bringing both graphs to a global coordinate system where they can be compared. In subsequent encounters, information can propagate between the two pose graphs, similar to the scenario of loop closures in a single session. The incorporation of anchor nodes makes efficient updates and quick optimization feasible.

As described by [Kim et al. \(2010\)](#), the anchor nodes allow us to estimate the offset between sessions. Moreover, they provide faster convergence to least-squares solvers and allow each session to optimize their poses before considering global constraints, such as from inter-session loop closures ([Ozog et al., 2016](#)).

This feature is advantageous for *long-term mapping* since it enables the production of the first consistent map of the environment when the data is gathered. Whenever a map containing a new session is constructed in a posterior period, and at least one encounter is detected, the anchor nodes allow the computation of the transformation that aligns this recent session with the previously acquired session. Subsequent inter-session loop closure detections will allow correction and improvement of both sessions.

Now that we conclude the introduction to the theory behind the selected method to align two or multiple sessions, in the following section, the latest State-of-the-art (SOTA) methods to achieve this alignment with a reference map, with particular emphasis in BIM models will be summarized.

3 Related research

This section will provide an overview of the state-of-the-art approaches that allow this alignment by using prior building information, such as BIM model, floor plans or point clouds, and methods supporting mapping.

3.1 Map-based 2D LiDAR localization and mapping

[Follini et al.2020](#) show how the standard Adaptive Monte Carlo Localization (AMCL) technique may be utilized to obtain the transformation matrix between the robot reference system and an extracted 2D map from the BIM model. They also state that the AMCL algorithm could overcome small objects that are not present in the BIM model due to the probability distribution of its beam model.

The same technique was applied by [Prieto et al. \(2020\)](#), [Kim et al. \(2021\)](#), [Karimi et al. \(2021\)](#), and [Kim and Peavy \(2022\)](#) to localize a wheeled robot in a 2D OGM produced from a BIM model. The primary distinction between these strategies is how they extract the OGM from the BIM model.

An OGM discretizes the environment into 2D square cells with a predetermined resolution; the value in each cell reflects the likelihood that an obstacle occupies the cell. Thus, an OGM allows distinguishing whether a space is free, occupied, or undiscovered.

Prieto et al. (2020) make use of the geometry of the spaces in the Industry Foundation Classes (IFC) file as well as the location and size of each opening, in contrast to Follini et al. (2020), who use the vertices of elements that intersected a horizontal plane and the Open CASCADE viewer to create an OGM in *pgm* format.

Karimi et al. (2020) created Building Information Robotic System (BIRS), an ontology that allows the generation and transfer of topological, semantic, and metric maps from a BIM model to Robot Operating System (ROS). An optimal path planner was included in the tool in (Karimi et al., 2021), incorporating crucial elements for the evaluation of the construction. However, this method still does not incorporate Mechanical, Electrical, and Plumbing (MEP) equipment.

A technique to transform an IFC file into a ROS-compliant Simulation Definition Format (SDF) world file appropriate for robot job planning was implemented by Kim et al. (2021). They evaluated their strategy for an automatic painting of interior walls. The prototype includes a converter that generates a ROS-compliant world file from IFC file and subprocesses that perform localization, navigation, and motion planning.

Later, a method to turn an IFC model into an Universal Robot Description Format (URDF) building environment was proposed by Kim and Peavy (2022) in order to add dynamic objects and for the purpose of door inspection. From this point, a robot may directly access lifecycle information from the BIM model for job planning and execution. Once they have the URDF model, they use PgmMap (Yang, 2018) to extract an OGM from it.

For 2D-LiDAR localization, Hendriks et al. (2021) propose a method that uses a robot-specific world model representation taken directly from an IFC file rather than from an OGM. In their factor graph-based localization strategy, the system receives information about the lines, corners, and circles in the immediate environment of the robot and builds data linkages between those items and the laser readings. They updated and assessed their approach for global localization in (Hendriks et al., 2022), producing superior results when compared to AMCL.

Boniardi et al. (2017) uses an architectural floor plan based on Computer-aided Design (CAD) rather than a BIM model. They use a GICP implementation for scan matching together with a pose graph SLAM system in their localization and mapping system. They transform a CAD floor plan into a 2D binary image and use it for robot localization in a wear-house-like scenario.

Later, they suggested an improved pipeline that outperformed Monte Carlo Localization (MCL) in the pose tracking problem for long-term localization and mapping in dynamic situations Boniardi et al. (2019).

In one of our previous contributions (Vega Torres et al., 2022), we proposed a method to create an OGM from a multistory IFC Model. Furthermore, we showed that the commonly used AMCL is not as resistant to change and dynamic environments as compared to Graph-based Localization (GBL) methods, such as Cartographer (Hess et al., 2016) and SLAM Toolbox (Macenski and Jambrecic, 2021). Based on these findings, we also offered an open-source approach that transforms OGM to Pose Graph-based Maps (PGBM) for reliable tracking of robot poses. This method was released to ease the transition of the localization algorithms from the classical Particle

Filter (PF) to more robust GBL methodologies, similar to what happened with the development of the SLAM algorithms.

3.2 Map-based 3D LiDAR localization and mapping

Other approaches investigated 3D LiDAR localization using 3D reference maps.

Gawel et al. (2019) presented a very accurate robotic building construction system. They use ray tracking with three *laser distance sensors*, a 3D CAD model, and a robust state estimator that merges IMU, 3D LiDAR, and wheel encoders to locate the end-effector with subcentimeter precision. They did this by taking several orthogonal range measurements while the robot was static.

In the technique proposed by Ercan et al. (2020) and Blum et al. (2020), the 3D LiDAR scan is aligned with the BIM model using the ICP algorithm.

While Ercan et al. (2020) limits the alignment to a few carefully chosen reference-mesh faces to overcome ambivalence, Blum et al. (2020) uses picture information to separate the foreground and background in the point cloud and uses only the latter for registration. The pipeline was then extended to provide a self-improvement semantic perception technique that can better handle environmental clutter and increase accuracy (Blum et al., 2021).

To take advantage of the high performance of Google Cartographer (Hess et al., 2016) for localization, Moura et al. (2021) suggest a method to create *.pbstream* maps from BIM models. Although this approach is quite practical, since they only employ Cartographer in localization mode, their method does not create a map of the environment if the robot is not localized and inside the boundaries of the reference map.

Oelsch et al. (2021) propose Reference-LOAM (R-LOAM), a technique that uses a combined optimization that includes point and mesh characteristics for 6 DoF Unmanned Aerial Vehicle (UAV) localization. Later, in (Oelsch et al., 2022), they improved their approach using pose-graph optimization to decrease drift even when the reference object is not visible.

A semantic ICP approach was presented by Yin et al. (2023). This method uses the 3D geometry and semantic data of a BIM model to achieve a reliable 3D LiDAR localization method. Their system suggests a BIM-to-Map conversion, turning the 3D model into a point cloud that is semantically enhanced. Their research demonstrates that a 3D LiDAR-only localization can be accomplished using an BIM model in uncluttered environments.

Another exciting strategy, suggested by Shaheer et al. (2022), relies on geometric and topological information in the form of walls and rooms rather than object semantics for localization. They build Situational Graphs (S-Graphs) using these data, which are subsequently used for precise pose tracking. Later, they improved their technique by allowing the acquisition of a map before localization, as well as the posterior matching and merging with an A-graph (extracted from BIM models). The combined map's ultimate designation was an informed Situational Graph (iS-Graph) (Shaheer et al., 2023).

Direct LiDAR localization (DLL) is a fast localization method introduced by Caballero and Merino (2021). They use a registration method based on non-linear

optimization of the distance between the points and a reference point cloud. Their method does not require feature extraction to achieve an accurate and fast registration. By correcting the anticipated pose using odometry, the technique can follow the robot’s pose with subdecimeter precision in real-time. Their technique performed better compared to AMCL 3D (Perez-Grau et al., 2017).

Numerous methods have been developed that use reference 2D and 3D maps for LiDAR localization and mapping. Most of them have concentrated on real-time localization without enabling pose-graph-based optimization approaches to provide a more accurate estimation of the calculated poses.

Additionally, practically every method requires the scanning to begin in a known initial pose that must be inside the boundaries of the reference map.

This requirement means that for several methods, there is no chance of localization or the generation of an aligned map if the robot starts from a location where the reference map is not visible or from where there are large Scan-Map deviations, like in a cluttered environment.

Furthermore, rather than retrieving a posterior accurate, updated, and extended map of the environment and detecting environmental changes, most researchers focused only on improving the accuracy of the pose-tracking process.

In this paper, we provide a strategy that addresses these problems and show that it is feasible to create an aligned, optimized map that is near the ground truth and discover changes in the environment.

4 Methodology

Our approach can be broken down into three key components, as shown in Figure 1. In **Step 1**, synthetic reference SD is generated automatically from large-scale 3D reference BIM models or point clouds.

Then, in **Step 2**, a real-world undistorted LiDAR SD acquired using a state-of-the-art LiDAR Inertial Odometry (LIO) algorithm is aligned and corrected using the reference 3D map.

Finally, in **Step 3**, the aligned map is further automatically analyzed, allowing the creation of an updated 3D map, which considers the detection of positive and negative environmental changes.

4.1 Map-based session data generation (Map to Session Data)

In this step, our objective is to encapsulate the geometry of the reference 3D map—whether it is a BIM model or a point cloud—into individual LiDAR scans with their corresponding feature descriptors. These descriptors serve to encode the visible geometry from the origin of the scan within the reference map, enabling us to rapidly find the correct alignment of real-world session data with a reference map.

In real-world data acquisition, Session Data (SD) refers to consecutive sensor data acquired from a particular place at different periods (Cramariuc et al., 2022). Nonetheless, since we aim to convert a reference map to synthetic SD, these data can be considered a set of LiDAR scans (with known carefully selected positions) and their corresponding descriptors.

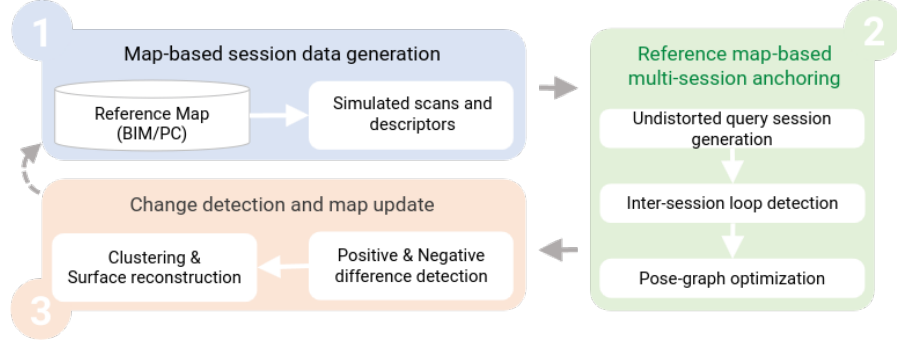


Fig. 1: Overview of **SLAM2REF**. The pipeline consists of three steps: map-based session data generation, Reference map-based multi-session anchoring, and Change detection and map update.

Formally, a session $\mathcal{S}$ is defined as follows:

$$\mathcal{S} := \left(\mathcal{G}, \{(\mathcal{P}_i, d_i)\}_{i=1, \dots, n} \right) \quad (3)$$

Here, $\mathcal{G}$ is a pose-graph map that contains the coordinates of the pose nodes, odometry edges, and optionally recognized intra-session loop edges with uncertainty matrices. These matrices represent how certain the positions of these edges are. This map can be saved in a text file, usually in *.g2o* format.

The $(\mathcal{P}_i, d_i)$ are the pairs of 3D LiDAR scans with their corresponding global descriptors of the i^{th} keyframe and n is the total number of equidistantly sampled keyframes.

Generating synthetic SD (simulated scans and descriptors) from a reference map can be subdivided into three substeps. First, an OGM is extracted from the reference map. This extraction is achieved in an automated manner, taking as input only the IFC model or the reference point cloud and the floor level (z coordinate value) from where the OGM should be generated. In a second substep, the OGM is used to find the poses in which the LiDAR scans will be simulated. In a third and final substep, LiDAR scans are rapidly simulated in the positions calculated in the previous step, and the corresponding descriptors are calculated.

These substeps have been optimized so that it is possible to efficiently simulate data from large-scale 3D BIM models and point clouds. The following subsections provide a more detailed explanation of each substep.

4.1.1 OGM from reference map

Initially, and for convenience, the 3D geometry of the reference map is reduced into a 2D OGM. This dimensional reduction has been demonstrated to be very computationally efficient, allowing the implementation of the pipeline in complex, large-scale models.

Moreover, a 2D OGM (with known scale and origin) allows the direct usage of the map with the ROS navigation stack for autonomous navigation (Macenski et al., 2023). Besides path planning, cost maps, and navigational algorithms, the ROS navigation stack includes several state-of-the-art features, such as the regulated pure pursuit algorithm to adjust the robot’s speed depending on the path with a particular focus on safety in constrained and partially observable spaces (Macenski et al., 2023).

The method for creating OGM varies based on the input data. Here, we outline how to do it for BIM models and point clouds.

4.1.1.1 OGM from IFC model (BIM2OGM)

The proposed automated generation of OGMs from BIM models builds upon prior work described in (Vega Torres et al., 2022). However, the key distinction lies in the enhanced automation of the pipeline.

For this purpose, we leverage the IfcConvert (IfcOpenShell Contributors, 2023a) tool and employ image-processing techniques akin to previous related works. IfcConvert, a command-line interface application within the open source IfcOpenShell project (Krijnen, 2015), facilitates the versatile conversion of a 3D BIM model from the *.ifc* file format to various other formats such as 3D meshes (*.obj*, *.dae*) or 2D layers (*.svg*). Detailed documentation for the IfcConvert functionality is available (IfcOpenShell Contributors, 2023b).

The input 3D IFC model is first converted to Scalable Vector Graphics (SVG) format and then processed with the OpenCV library to output different layers as Portable Network Graphics (PNG) files. These layers will then be merged to produce the final Portable Gray Map (PGM).

To ensure compatibility with the ROS navigation stack and facilitate accurate scan simulations, the 2D PGM map must adhere to specific guidelines. It should represent unknown (external) regions in gray, navigable space (floor) in white, and potential collision-causing objects (e.g., walls and columns) in black.

IfcConvert is used to convert the 3D IFC model into 2D SVG files with the desired elements intersecting a plane at the chosen height. Furthermore, the resolution and size of the output SVG image are modified to only include the elements of interest.

To generate the OGM, we leveraged the semantics of the BIM model, focusing on extracting permanent elements like walls, ceilings, columns, and floors. This process excludes non-permanent features and objects invisible to LiDAR sensors, such as spaces, doors, windows, and curtain walls.

Filtering just permanent structural information about the building enables finding reliable correspondences between the geometry from the BIM model and real-world 3D LiDAR data. In this letter, we assume that the permanent structures in the BIM model are reliable features for localization and scan-matching. In the presence of open doors and windows, their exact placement in the space is unknown (open, closed, or semi-open) and is not provided in the BIM model; therefore, those elements should not be considered while creating the 2D OGM or any source of information used for alignment or localization.

A critical consideration in the conversion of SVG files to PNG format is the choice of units utilized within the original SVG file. By default, IfcConvert assigns millimeters

as the unit of measurement for the SVG files. However, these millimeters do not undergo a direct one-to-one transformation to pixels during the conversion to PNG. Consequently, it becomes imperative to eliminate explicit unit specifications within the SVG file to ensure consistent scaling and preservation of the established coordinate origin during the conversion to PNG.

Additionally, it is critical to consider the effect of displacement while creating sections at different heights. While the scale will be maintained, the values of the coordinates of the geometry (saved in *paths*) in the SVG file will be adjusted according to the elements that intersect that specific height. To counteract this effect and have all the PNG images in the same coordinate system, the images are shifted according to the x and y values saved in the *data matrix* of the SVG generated with IfcConvert.

Automating the creation of the OGM involves producing the following two sections:

1. In the indoor layer, the floor area is designated as white. This section is generated at the z-coordinate corresponding to the upper surface of the slab of interest, i.e. at the floor level where the alignment should happen. Subsequently, the resultant gray-scale PNG image from this SVG is converted to binary. Then, its inverted version represents the indoor layer, in which the floor is represented as white pixels and the rest as black.
2. In the collision layer, we extract permanent elements like walls and columns while excluding non-permanent structures such as doors and windows. The creation of this layer occurs slightly above (1 m) the z-coordinate of the previous layers. It is crucial to note that the coordinate system of this image deviates from the preceding layer due to its creation at a different height. Therefore, it is imperative to compensate for this offset, as previously explained, before converting it into PNG format.

Subsequently, the indoor layer is placed over a gray image of the same size, allowing to distinguish outdoor (unknown) and indoor areas.

Finally, the pixels in the black color of the collision layer are transferred to the indoor layer. Given this, the final OGM is created and saved in the rasterized ROS standard PGM format. Figure 2 illustrates the layers and the final 2D map.

Additionally, a corresponding YAML Ain't Markup Language (YAML) configuration file is generated, containing crucial details such as the origin and resolution of the 2D map, extracted from the *data-matrix* of the initial SVG file.

Besides being an essential step in our pipeline, accurately creating a 2D OGM holds significant potential for SOTA localization algorithms, facilitating rapid and collision-free autonomous navigation. This has been exemplified by [Vega Torres et al. \(2022\)](#) and corroborated by numerous other studies (refer to Section 3).

4.1.1.2 OGM from a point cloud

The steps involved in creating a OGM from a point cloud are as follows: First, a 2D grid is created to the length and width of the point cloud and scaled given a grid resolution. Each cell within this grid is initially assigned a gray color.

Then, and as discussed in ([Vega Torres et al., 2021](#)), the points are projected onto the XY plane, considering the resolution of the grid and its origin (the minimum XY

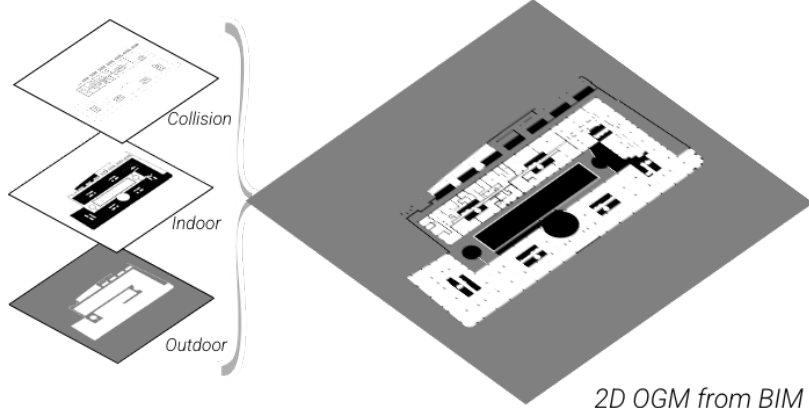


Fig. 2: Generated OGM from the BIM model. On the left, the different layers, and on the right, the merged final OGM.

coordinate of the point cloud). If points within a cell are found to be near the floor level (within a range of ± 0.5 m), the cell is colored white, signifying navigable space.

On the other hand, cells are colored black if points are detected at a height 1 m above the floor level, assuming that this region predominantly consists of walls, columns, and other permanent elements.

4.1.2 Locations for data simulation

Once a correct OGM is generated from the reference map, this is utilized to find proper locations where LiDAR scans will be simulated. These locations should be equally separated coordinates ordered by proximity, aiming to closely replicate real-world data acquisition with full coverage of the map. To this aim, we first extract the skeleton of the image, which gives a smooth path similar to the one a person would follow during acquisition with a mobile LiDAR or scanning device. Then, points are sampled over this path in a uniform manner.

Similarly, as proposed in (Vega Torres et al., 2022), the process extracts a skeleton from the OGM. This skeleton is derived using the approach outlined by Lee et al. (1994), producing a smooth trajectory over the free space that interconnects all rooms and open areas within the OGM.

In a previous version of our pipeline (Vega Torres et al., 2023), we used a Wavefront Coverage Path Planner (PP) (Zelinsky et al., 1993) over this skeleton to find the waypoints in which the 3D LiDAR will be simulated. However, the Wavefront Coverage PP approach is inherently intricate, making it unfeasible to be applied over large OGMs without consuming large amounts of computational resources.

Therefore, to handle large-scale reference maps, we propose the following method instead, which tries to sample uniformly key points over the path created with the skeleton approach:

(1) The scan locations are initially extracted using image processing techniques. This involves generating masks with equally spaced vertical and horizontal lines, isolating only the white pixels intersecting these masks and the previously generated skeleton. The idea behind this is that only isolated pixels will remain rather than elongated lines present in the skeleton.

(2) Subsequently, the corresponding center points of the remaining pixels are extracted using a contour detection algorithm. To ensure a minimum distance between points, the spatial distribution of these coordinates is downsampled.

(3) Finally, the coordinates are sequentially ordered using the nearest neighbor algorithm.

Figure 3 shows the calculated scan locations for an OGM of a large building.

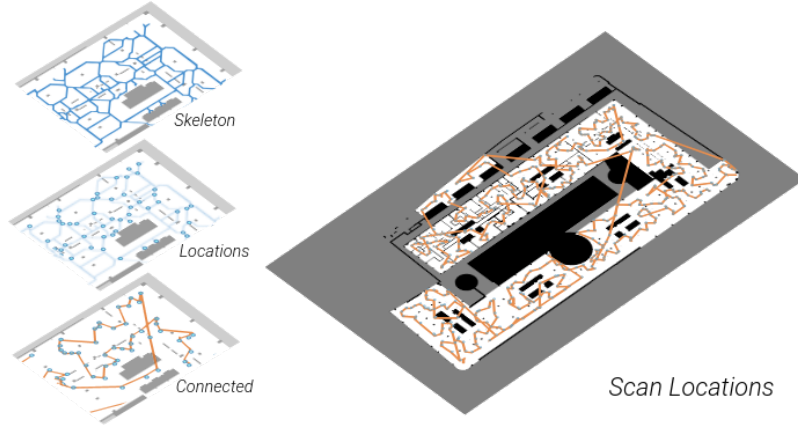


Fig. 3: Calculated locations for scan simulation. On the left are the main steps, and on the right are all the calculated positions in the entire OGM.

4.1.3 LiDAR data simulation

In our previous work (Vega Torres et al., 2023), we utilized the identified waypoints to set navigational goals for a robot operating autonomously within the ROS navigation stack, simulated in the Gazebo physics engine (Koenig and Howard, 2004). Then, a sequence of simulated 3D LiDAR scans was produced with Gazebo and saved in rosbag files. Here, we present an enhanced approach that eliminates the need for ROS or Gazebo; by such means, we avoid the creation of large rosbag files containing redundant information.

Instead, we propose leveraging Blender Sensor Simulation Toolbox (BlenSor), a versatile software designed for simulating various range scanners (Gschwandtner

et al., 2011; Gschwandtner, 2013). With the BlenSor Application Programming Interface (API), we can automatically load the coordinates for simulating LiDAR scans (calculated in the previous step), streamlining the simulation process.

The process of simulating LiDAR data can be subdivided into three main steps:

1. The reference map is converted to an stereolithography (STL) mesh. In the case of a Building Information Modeling (BIM) model, this involves conversion to Wavefront .obj file (OBJ) format after filtering only permanent structures using IfcConvert, similar to the process employed in creating the 2D OGM. However, instead of generating an SVG file, our method creates an OBJ file containing the 3D geometry of the model described explicitly. To ensure precise 3D conversion, our approach selectively *includes* required permanent elements (e.g., walls, columns, floors, and slabs) rather than *excluding* entities. Our experiments revealed that the *exclusion* command does not consistently produce satisfactory results for this 3D conversion. Subsequently, the generated OBJ file is converted to STL format for seamless integration of the geometry into BlenSor.

When dealing with a point cloud as the reference map, the ball pivoting method has consistently demonstrated reliability in reconstructing mesh surfaces from 3D point clouds. Before applying this method, the process involves estimating the normals of the point cloud and calculating an optimal radius based on the average nearest neighbor distance, facilitating accurate and efficient surface reconstruction.

2. Later, the coordinates determined in the preceding steps, where the data will be simulated, are transformed from pixels (in 2D) to meters (in 3D). This conversion utilizes the scale and origin information specified in the YAML file of the corresponding OGM.
3. Subsequently, the simulated LiDAR properties are adjusted to align with those employed in real-world scanning. Then, a sub-process initiates the parallel simulation of 360° LiDAR scans at these coordinates using BlenSor.

Finally, and after the simulation, Scan Context (SC) descriptors are created for each simulated scan. More information about these descriptors will be provided in the following section 4.2.1 (Step 2.1).

Following the steps above, the geometry of the reference map or the permanent objects in the BIM model is now established as a reference session, denoted as $\mathcal{S}_{\mathcal{R}}$, and is illustrated in Figure 4.

In the subsequent step, this synthetic Session Data, encompassing descriptors and simulated scans, will be leveraged for fast place recognition and data alignment. However, before this process, it is necessary to generate session data from real-world datasets.

4.2 Reference map-based multi-session anchoring

To derive a globally consistent map aligned with the reference map from real-world sequential LiDAR data, the following three substeps are executed: (1) Creation of the real-world motion-undistorted query session $\mathcal{S}_{\mathcal{Q}}$, which is similar to the synthetic reference session $\mathcal{S}_{\mathcal{R}}$ (created as explained in the previous section); however, from real-world data. (2) Place recognition for inter-session loop detection between $\mathcal{S}_{\mathcal{Q}}$ and $\mathcal{S}_{\mathcal{R}}$.

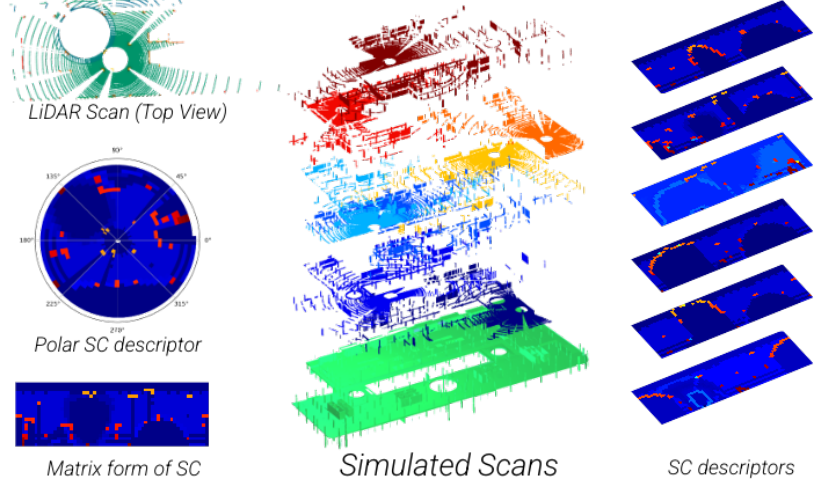


Fig. 4: Synthetic session data from the reference map. On the left, from top to bottom: Top view of one LiDAR scan, its corresponding polar scan context (SC) descriptor, and the descriptor in the matrix form. In the middle, a set of simulated scans and the STL mesh from the BIM model are used. Right, corresponding SC descriptors for the simulated scans.

(3) Pose graph optimization with multi-session anchoring and pose refinement with KNN loops and a final ICP registration. These substeps are described in detail in the following subsections.

Figure 5 illustrates a flowchart outlining the complex multi-session anchoring process in the **SLAM2REF** framework.

Following the generation of session data (SD) from the reference map $\mathcal{S}_R$ (Step 1 presented in Section 4.1) and the construction of the real-world query session $\mathcal{S}_Q$ (Section 4.2.1), the alignment procedure can be initiated. This involves an inter-session loop detection phase employing Indoor Scan Context and YawGICP (Section 4.2.2), which identifies encounters $\mathbf{c}$ denoting correspondences between the sessions. These encounters, along with initial odometry constraints, are integrated into a factor graph problem. Subsequent to optimization, pose refinement is carried out using KNN loops (Section 4.2.3) and a *final ICP* process. The resulting information comprehends the following elements attributed to the query session: the anchor node Δ_Q^* , which facilitates the global alignment to the reference map, the optimized 6-DoF poses of each scan $\mathbf{x}_Q^*$, and a confidence level list ν_Q providing the reliability of each pose after scan registration.

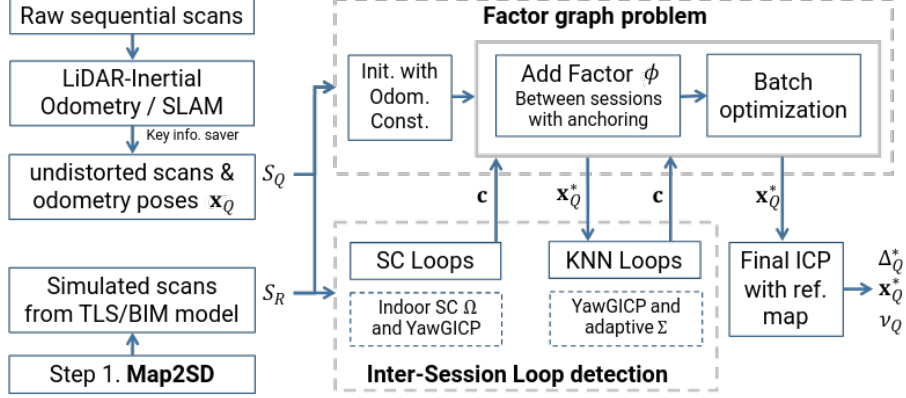


Fig. 5: Comprehensive flowchart illustrating the multi-session anchoring process within **SLAM2REF**. This process includes the generation of session data from the reference map S_R , creation of the real-world query session S_Q , inter-session loop detection using Indoor Scan Context and YawGICP, and pose refinement with KNN loops and *final ICP*. The outcome includes the anchor node Δ_Q^* , optimized 6-DoF poses x_Q^* , and a confidence level list ν_Q for each pose in the query session.

4.2.1 Creation of the real-world query session

The correct generation of a query session S_Q from real-world data involves three primary substeps, elaborated upon as follows.

4.2.1.1 Motion distortion correction

Point clouds acquired from mobile spinning LiDAR sensors often experience motion distortion because the rotating laser array collects points in various instances during a sweep, leading to inaccuracies. Therefore, one of the main issues using LiDAR-only algorithms is the difficulty in correcting motion-distorted LiDAR scans in the presence of fast motion.

In some SOTA LiDAR-only SLAM algorithms, the authors have assumed constant velocity models to overcome this issue, as done in KISS-ICP (Vizzo et al., 2023). Although this assumption can hold for data acquired with LiDAR placed over autonomous cars and simplistic motion patterns, as in the KITTI raw dataset (Geiger et al., 2013), the constant-velocity model cannot capture subtle movements and generally does not hold for data acquired with handheld devices or Unmanned Vehicles (UVs) in indoor or outdoor scenarios (Zheng and Zhu, 2023).

Therefore, we take advantage of the motion distortion correction (MDC) of one SOTA LIO system to generate undistorted scans before alignment with the reference map.

In particular, we leverage the MDC implementation in Direct LiDAR Inertial Odometry (DLIO) (Chen et al., 2023), which, inspired by Forster et al. (2016), generates continuous-time trajectories. Their approach considers a motion model characterized by constant jerk and angular acceleration compensated with IMU measurements. This enables fast and parallelizable point-wise motion correction.

Once the scans are deskewed with the information from the IMU, keyframe scans can be extracted with timestamps and odometry calculated poses. This process is explained in the subsequent section.

4.2.1.2 Key information saver

The goal here is to save equally spaced undistorted scans (i.e., after a specific variation of time, translation, or rotation) with respective odometry estimated poses from a sequence of data that was previously recorded in a ROS *bagfile* during acquisition with a mobile mapping system device.

To extract keyframes and construct the real-world query session $\mathcal{S}_Q$, the methodology proposed by Kim et al. (2022) presents a viable approach. The authors implemented loop closure mechanisms and keyframe information-saving capabilities as an extension in several SOTA algorithms.

In general, the approach can vary depending on the available data. When dealing with LiDAR-only data, SC-A-LOAM (Kim et al., 2022), an enhanced version of A-LOAM (Zhang and Singh, 2014) is a valid technique; however, it assumes constant velocity for MDC. For an additional calibrated 9-axis IMU, the corresponding enhanced version of LIO-SAM (Shan et al., 2020) can be used.

If we deal with 9-axis or only 6-axis IMU measurements, which are typical for the internal IMUs of LiDAR and camera sensors, our open source keyframe information saver² together with almost any LIO pipeline can be used (e.g., FAST-LIO2 (Xu et al., 2022), FASTER-LIO (Bai et al., 2022) or iG-LIO (Chen et al., 2024)). Something essential to consider is that the LIO pipeline should publish (i.e., make available) the ROS topic with the undistorted scan in the local coordinate system. This last characteristic is not standard and depends on the used MDC strategy.

Since DLIO showed the best MDC results in our experiments, we implemented and made open-source the corresponding enhanced version that transforms the deskew scan to the correct local pose after undistortion².

After saving the keyframe scans along with odometry information (i.e., time-stamped approximate 6-DoF poses), the final step to generate the query session involves feature descriptor extraction to encode the geometric information of the scans. This process will facilitate efficient comparison with reference session descriptors later.

4.2.1.3 Indoor Scan Context descriptor

For place recognition, we introduce the new Indoor Scan Context Descriptor (ISCD). This variant diverges from the original Scan Context descriptor by focusing exclusively on indoor scans, as opposed to outdoor scans typically encountered in autonomous car environments, for which SC was originally conceived. With ISCD, our objective

²<https://github.com/MigVega/Key-Info-Saver-SLAM>

expands beyond merely eliminating ceiling points, which are notably common in indoor scans, especially in acquisitions with significant variations in pitch and roll angles, as usually encountered in handheld systems. Moreover, we aim to selectively filter permanent vertical building elements perpendicular to the XY-plane, characterized by visible vertical surfaces of considerable length.

Inspired by [Kim and Kim \(2018\)](#); [Kim et al. \(2021\)](#), and by the formal definitions in [\(Wang et al., 2020; Li et al., 2021\)](#), the creation of ISCD is as follows: Azimuthal and radial bins split the 3D scan from the top view following an equally spaced arrangement (for reference, see an example on the left side of Figure 4).

In the Cartesian coordinate system, we defined a LiDAR scan with n points as $\mathcal{P} = \{\mathbf{p}_1, \mathbf{p}_2, \dots, \mathbf{p}_n\}$ with each point $\mathbf{p}_k = [x_k, y_k, z_k]$. Each point p_k can be converted into a polar coordinate system, as follows:

$$\begin{aligned}\mathbf{p}_k &= [r_k, \theta_k, z_k], \\ r_k &= \sqrt{x_k^2 + y_k^2}, \\ \theta_k &= \arctan \frac{y_k}{x_k}.\end{aligned}$$

The point cloud is then segmented into N_s sectors and N_r rings by equally dividing polar coordinates in azimuthal and radial directions. Each block is represented by:

$$B_{ij} = \left\{ \mathbf{p}_k \in \mathcal{P} \mid \frac{(i-1) \cdot R_{\max}}{N_r} \leq r_k < \frac{i \cdot R_{\max}}{N_r}, \right. \\ \left. \frac{(j-1) \cdot 2\pi}{N_s} - \pi \leq \theta_k < \frac{j \cdot 2\pi}{N_s} - \pi \right\},$$

where $i \in [1, N_s]$, $j \in [1, N_r]$, and $R_{\max}$ is the maximum radius considered to create the descriptor. In contrast with the original Scan Context Descriptor (SCD), instead of taking only the z value of the highest point in the bin b_{ij} , in ISCD, we only assign a value equal to 1 if there are a minimum of $ISC_{\min}$ points in the bin, and 0 otherwise. Formally:

$$b_{ij} = \begin{cases} 1 & \text{if count}(\mathbf{p}_k \in B_{ij}) \geq ISC_{\min} \\ 0 & \text{otherwise} \end{cases}$$

The final ISCD $\Omega \in \mathbb{R}^{N_r \times N_s}$, can be generated by:

$$\Omega(i, j) = b_{ij}.$$

The global signature Ω is a 2D matrix that efficiently encodes the geometry of mainly permanent elements (e.g., walls and columns) visible from the position of the sensor.

Note that if $B_{ij} \in \emptyset$, $\Omega(i, j) = b_{ij} = 0$, i.e., if in the bin there are no scan data because the bin is free or occluded, the bin will have a value of zero and will be visible as a blue color in the image representation of the descriptor (as shown in 4 and 6).

In the following section, these descriptors are exploited to rapidly determine the rough alignment between the query and reference sessions.

4.2.2 Place recognition for inter-session loop detection

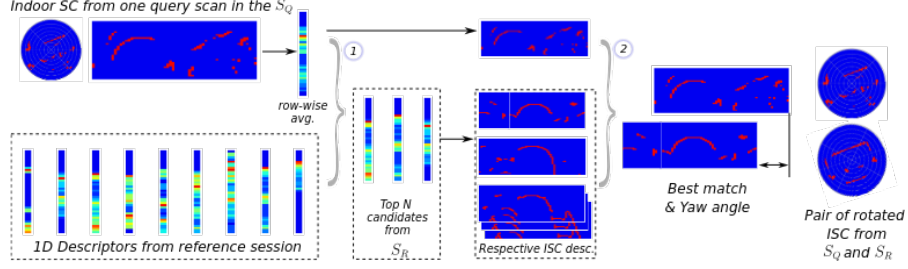


Fig. 6: Indoor Scan Context loop detection: The query session’s scan is converted into 1D rotational invariant descriptors. These descriptors are quickly compared with those from the reference session to select the top N_c candidates (see number 1). In the second phase, the 2D descriptors of these candidates are compared using cosine similarity while systematically varying the column position to identify the best match and optimal yaw angle alignment.

Having S_Q (real-world query session) and S_R (session from the reference map), we aim to align these two sessions. With this aim, we look for correspondences comparing the previously generated ISCDs between the sessions to find inter-session loop closures. This task is also known as *place recognition*, in which one aims to identify or determine the specific location or place of sensor measurements (in our case, single LiDAR scans) within a given map.

In order to facilitate quick comparison, the 2D descriptor is condensed into a one-dimensional vector. This vector is generated by calculating the average of the rows in the 2D descriptor. This average ensures rotation invariance, meaning that if a scan is in a location that is approximately the same but with a different yaw angle, the resulting 1D descriptor will remain unchanged.

The comparison between the query scan (from S_Q) and the scans from the S_R is facilitated by employing a K-nearest neighbors (KNN) search in a KD-Tree and using the L2-norm metric.

Subsequently, the corresponding 2D descriptors of the N_c closest 1D descriptors are compared using the *column-wise cosine distance*.

This column-wise cosine distance is calculated to identify the similarity between two ISCDs Ω^q and Ω^r . Let $\mathbf{v}_i^q$ and $\mathbf{v}_i^r$ be the i^{th} column of Ω^q and Ω^r ; the score can be found by:

$$\varphi_i(\Omega^q, \Omega^r) = \frac{1}{N_s} \sum_{i=0}^{N_s-1} \left(\frac{\mathbf{v}_i^q \cdot \mathbf{v}_i^r}{\|\mathbf{v}_i^q\| \cdot \|\mathbf{v}_i^r\|} \right).$$

A comparison conducted column by column is beneficial for handling dynamic entities or slight differences between the reference map and the query session (e.g., new

furniture or clutter) since although some columns of the 2D descriptor may show variations, the remaining columns will exhibit similarities. However, relying solely on this comparison overlooks the possibility of revisiting the exact location from a different perspective. To tackle this limitation and ensure rotational invariance in the matching process, the method computes distances using a range of column-shifted scan contexts. Then, it identifies the shift that yields the minimum distance. This procedure resembles the coarse alignment of two sets of points, focusing mainly on aligning the yaw angle. By implementing this approach, the optimal number of column shifts (i.e., optimal yaw angle) for alignment and the corresponding minimum distance can be determined.

Formally, if we compare Ω_k^q and Ω^r where Ω_k^q is Ω^q shifted by k^{th} column. The final score is calculated as follows:

$$\Phi_i(\Omega^q, \Omega^r) = \underset{k}{\operatorname{argmin}} \varphi_i(\Omega_k^q, \Omega^r).$$

The matched pairs are subsequently refined through a filtering process employing an empirical threshold, denoted as ϵ , applied to the calculated minimum distance metric, Φ_i .

After detection of Indoor Scan Context (ISC) loop closures, a 6D relative constraint is established between two keyframes if there is a successful alignment between a sub-map from the reference session, denoted as $\mathcal{P}_{R,i}$ (which comprises the three closest scans to the one that matched the scan in the query session), and the single undistorted scan from the query session, denoted as $\mathcal{P}_{Q,j}$.

The correctness of the alignment between these two keyframes is essential for the subsequent steps in the pipeline, as it dictates the effectiveness of the initial global registration between sessions.

To achieve this alignment in a robust way, we introduce YawGICP, an improved variant of the GICP algorithm. YawGICP primarily focuses on translational changes and yaw angle adjustments, thereby mitigating significant pitch or roll rotations commonly induced by conventional GICP alignment procedures. This precaution prevents instances where standard GICP may accidentally rotate the source point cloud by 90 degrees (in pitch or roll), leading to erroneous associations between wall, ceiling, or floor points.

The YawGICP is initialized with the yaw angle calculated in the previous step.

Consistent with prior work (Vega Torres et al., 2023; Kim and Kim, 2022), only ISC loops exhibiting a satisfactory fitness score, indicating a high percentage of inliers, are considered. These loops are then incorporated into the factor graph problem with low covariance Σ_c , serving as factors between sessions with anchoring. Further elaboration on the factor graph problem will be provided in the subsequent section (4.2.3). Figure 7 illustrates the detected ISC loop closures, which are then classified into correct and incorrect using YawGICP.

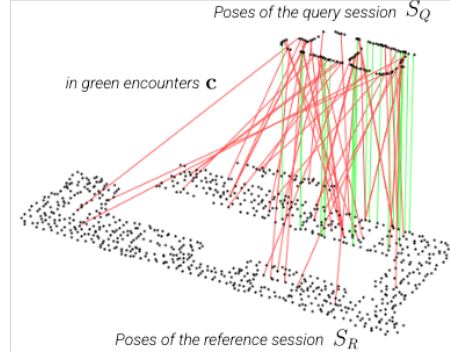


Fig. 7: Detection of Indoor Scan Context loop closure between sessions. On the top are the poses from the query session, and on the bottom are the poses from the reference session, in this case, created from a BIM model. Correct correspondences are represented by green lines, while erroneous ones as red. After the YawGICP step, the erroneous correspondences are effectively discarded.

4.2.3 Pose graph optimization and data alignment

In this substep, the initial odometry constraints derived from the preserved session data (referenced in substep 4.2.1) and previously identified inter-session ISC loop closures (introduced in substep 4.2.2) are leveraged to achieve the data alignment.

The objective is to first roughly align the entire query session with the reference session from the reference map. Consequently, even if some scans within the query session’s keyframes do not have any correspondence with the reference session, they are still aligned to the most cohesive pose based on the identified correspondences (SC loops) with adjacent scans and the provided odometry constraints.

Formally, in this contribution, the alignment between the sessions is done using *multi-session anchoring*. This method was originally introduced by Kim et al. (2010) and was further developed by McDonald et al. (2013), Ozog et al. (2016), Kim and Kim (2022). One of the main motivations behind these projects is to solve the so-called *multi-robot mapping* problem. In this context, and as explained in Section 2, maps generated by different robots commonly have distinct reference coordinate systems, which require the merging of these maps to form a globally consistent map with a unified global coordinate system.

We formally define our problem as follows: Given two sessions, S_Q and S_R , each provided with odometry constraints, and in the case of S_Q , potentially equipped with intra-session loop closure constraints identified by a SLAM algorithm with a key information saver (as explained in Section 4.2.1), our objective is to determine the optimal poses for the nodes in S_Q . These poses should effectively align the measurements within S_Q with those of S_R , considering the existence of inter-session loop closure constraints between the two sessions.

As explained in 2, multi-session anchoring can be formulated as a factor graph MAP optimization problem.

To properly consider the encounter measurements ($\mathbf{c}$) in the MAP formulation in Eq. (2), we need to redefine the relative measurement model $h(\cdot)$ in the global frame with the help of the anchor nodes.

This adjustment is needed, considering that the encounter is a global assessment between two trajectories. However, the pose variables for each trajectory are defined in the session's local coordinate frame. With the anchor nodes, the poses of the respective sessions are transformed into a global frame, where a comparison with the measurement becomes possible.

The measurement model $h(\cdot)$ is modified to $h'(\cdot)$, to incorporate the anchor nodes, and therefore, the respective term in Eq. (2) is changed to:

$$\sum_{j \in N_e} \|h'_j(\mathbf{x}_{R,j}, \mathbf{x}_{Q,j}, \Delta_Q, \Delta_R) - \mathbf{c}_j\|_{\Sigma_c}^2$$

The difference $\mathbf{c}$ in the global frame between a pose $\mathbf{x}_R$ and a pose $\mathbf{x}_Q$ is estimated by $\mathbf{c} = (\Delta_R \oplus \mathbf{x}_R) \ominus (\Delta_Q \oplus \mathbf{x}_Q)$, where $\oplus$ and $\ominus$ are the SE(3) pose composition operators (Smith et al., 1990; Blanco-Claraco, 2021).

The operation $\Delta_Q \oplus \mathbf{x}_Q$ represents concatenating the transformation of $\mathbf{x}_Q$ (the second pose) to the reference system already transformed by the anchor node Δ_Q . In SE(3), the operator $\oplus$ is equivalent to matrix multiplication (Blanco-Claraco, 2021).

Hence, the subsequent *factor between sessions with anchoring* will properly integrate the encounters in the pose graph optimization. It achieves this by initially transforming the poses of each session into the global frame using the anchor nodes.

$$\begin{aligned} & \phi(\mathbf{x}_{R,i}, \mathbf{x}_{Q,j}, \Delta_R, \Delta_Q) \\ & \propto \exp\left(-\frac{1}{2} \|((\Delta_R \oplus \mathbf{x}_{R,i}) \ominus (\Delta_Q \oplus \mathbf{x}_{Q,j})) - \mathbf{c}\|_{\Sigma_c}^2\right) \end{aligned} \quad (4)$$

While initializing the factor graph, the odometry constraints from both sessions and the constraints after ISC loop detection are added to the optimization problem, the first as *between factors* and the latter as *factors between sessions with anchoring*.

Considering that in our scenario, our objective is to use the coordinate system of $\mathcal{S}_R$ as the global system for alignment, the anchor node Δ_R of the reference session should be assigned an insignificantly small covariance (Σ_P). Conversely, for the anchor node Δ_Q of the query session, a significant covariance is assigned (Σ_L).

Moreover, the odometry poses are also added to the factor graph. However, since $\mathcal{S}_R$ comes from the reference map, its poses $\mathbf{x}_R$ are treated as fixed and should not be altered by the optimization. To avoid changes to these poses, they are added to the factor graph optimization problem as *prior factors* with very low covariance (Σ_P) in its noise model.

Following batch optimization, the intermediate optimized values of the anchor node Δ_Q^* and the poses $\mathbf{x}_Q^*$ are obtained. However, these poses are expressed in the local coordinate system of $\mathcal{S}_Q$. To convert them from this local coordinate system (denoted as ${}^Q\mathcal{G}_Q^*$) to the global coordinate system ${}^W\mathcal{G}_Q^*$ of the reference map, we apply the

following transformation to each pose $\mathbf{x}$ in the graph:

$${}^W\mathbf{x}_Q^* = \Delta_Q^* \oplus {}^Q\mathbf{x}_Q^*,$$

where W is the global coordinate system, or in our case, the coordinate system of the reference session.

After the previous step, the query session roughly aligns with the reference session. To further refine the poses of the query session, we introduce a rapid KNN loop detection method with adaptive covariance. Initially, submaps are generated by selecting KNN scans from the scan to be aligned within the query session, along with the k-nearest scans from the reference session. Subsequently, the YawGICP algorithm (see 4.2.2) is employed to register these two submaps, and the quality of registration is assessed based on a predefined fitness threshold, classifying the alignment as either good, acceptable, or unacceptable.

Upon acceptance of the alignment, the constraints are added to the optimization problem as factors between sessions with anchoring with adaptive covariance. This adaptive covariance strategy assigns a very low covariance in the noise model to constraints originating from well-registered keyframe submaps, while constraints from just acceptable registrations receive a higher covariance. This approach allows the pose graph optimization to appropriately weigh the influence of these constraints in calculating optimized poses.

After conducting batch optimization one more time with incorporated odometry, ISC, and KNN constraints in the factor graph problem, the resulting poses undergo further refinement through a *final ICP* registration. Unlike previous steps that relied on registration with simulated scans from the reference map, this stage utilizes a one-centimeter-dense point cloud obtained from the reference map as the registration target. In case the reference map is a BIM model, this point cloud is created by sampling uniformly points over a mesh of permanent elements in the building (i.e. without doors and windows similarly as done in Step 1, section 4.1.1)

Due to the high density of the target point cloud, GICP fails to offer any significant advantage over Point-to-Point (P2P)-ICP (Besl and McKay, 1992). In fact, in specific scenarios, GICP yields inferior results. Therefore, we have opted to use P2P-ICP, which not only produces competitive results but also operates considerably faster.

To speed up computations and avoid the time-intensive KNN search associated with registrations involving a large target point cloud, scans within the query session are allocated into proximity-based groups. Subsequently, for each group, a target point cloud is created, dynamically cropping the reference map into spheres. The individual source scans within each group are then registered concurrently, leveraging parallel computing techniques.

The registration results are evaluated using three metrics. One metric is the Root Mean Square Error (RMSE), and the other two correspond to fitness scores calculated at two distinct maximum P2P distances: F_1 and F_2 . The fitness score is the percentage of source inliers, considering a maximum P2P distance threshold to classify points as inliers after registration.

These metrics are computed explicitly for points located within 30 cm from the target point cloud after registration. This approach ensures the exclusion of points

outside the reference map or those influenced by significant environmental changes, such as the addition of new walls or large pieces of furniture.

Depending on the metric values, the resulting aligned scans are categorized into four classes: Perfect, Good, Bad, and Outside the Map. The result is saved on a list, denoted as ν_Q .

The resulting poses will be used in the subsequent step to create the final aligned map and compare it accordingly with the reference map.

4.3 Change detection and map update

Following the completion of the prior steps, the two sessions have been precisely aligned, and they now share a unified coordinate system. Subsequently, a comprehensive 3D map of the most up-to-date environmental state can be generated by placing the keyframes $\mathcal{P}_{Q,i}$ from the query session $\mathcal{S}_Q$ in the estimated poses ${}^W\mathbf{x}_{Q,i}^*$, which are now in the global coordinate system.

If desired and to ensure the integrity and fidelity of the final map representation, it is recommended to exclusively incorporate scans classified as "perfectly" or "good" aligned within ν_Q during the map construction process.

However, it is essential to note that although the remaining poses may not meet the strict alignment criteria with the reference map, they have already undergone significant optimization through odometry and loop closure constraints. Consequently, they can be utilized to generate the final map and even extend the reference map if the scan extends beyond its boundaries.

Since both maps are now aligned, a comparison of the two 3D maps becomes feasible. The comparison process involves categorizing the elements in the map into three distinct types: Positive differences (PDs) denote instances where new objects have been introduced compared to the reference map; negative differences (NDs) signify the removal of objects previously documented in the reference map; and unaltered elements (UEs) denote features that remain constant across both maps.

This categorization is facilitated with the OctoMap library (Hornung et al., 2013). OctoMap, a widely-used library in robotics and 3D mapping, operates by dynamically updating voxel occupancy status within its octree structure as new point clouds are integrated. The analysis of measurement densities in OctoMap enables us to distinguish between occupied and free space, facilitating reliable 3D mapping.

Additionally, we also leverage the probabilistic capabilities of OctoMap during measurement accumulation to facilitate the automatic removal of dynamic elements from the final point cloud. This removal is done based on occupancy patterns across multiple scans. The resulting map is the one used to detect PDs and UEs in the preceding step. Moreover, OctoMap calculates free space by identifying regions where the sensor fails to detect objects; this free space will be leveraged for NDs detection later.

To detect PDs and UEs, a P2P distance threshold is used between a point cloud from the reference map (also used in the previous final ICP step) and the newly created map with OctoMap, similar to what was presented in (Vega Torres et al., 2023). A signed distance computation allows the distinction of points that are near and far from

the reference map. Near points allow for the confirmation of UEs, whereas distant points are regarded as PDs.

The point cloud of identified PDs is passed through an outlier removal process. Subsequently, the point cloud undergoes a segmentation process through the density-based clustering technique (DBSCAN). This step is based on a neighbor-distance threshold and a minimum number of points per cluster.

Lastly, for each Positive Differences (PD) cluster, a mesh is created using cubes from a Voxel Grid (VG) of the point cloud.

Voxels, in contrast to other surface reconstruction approaches, capture the actual geometry of objects present in the scene. This leads to improved visualization of the new elements in conjunction with the reference map, providing a better understanding of the scene.

The process of detecting NDs involves conducting a visibility analysis using individual scans from the query session ($\mathcal{P}_{Q,i}$). As mentioned before, the OctoMap library facilitates this analysis by calculating the free space, i.e. areas where the LiDAR did not detect any objects from its origin point. Similarly, as with the PDs, this free space is used together with a P2P distance threshold against a point cloud sampled from the reference map to identify the NDs.

The regions at the intersection between the reference map and the free space are the NDs. These are then passed through the outlier removal and clustering process, removing isolated points and small clusters.

The final voxels are transformed into meshes and are colored blue for PDs and red for NDs. An exemplary result is depicted in 8.



Fig. 8: Positive and negative differences between the point cloud and the reference BIM model are illustrated as follows: (a) A picture of the real-world scene. (b) Visualization of the detected changes in the form of voxelized clustered meshes with positive differences depicted in blue and negative differences in red. Particularly, it is visible that the windows in the model are smaller compared to the real-world windows.

5 Experiments

In this section, we present the data used to evaluate the efficacy of the proposed strategies. Comprehensive implementation details, such as the values of the essential parameters, are meticulously outlined to ensure a thorough understanding of our approach.

5.1 ConSLAM dataset

To ensure reproducibility and benchmarking, we evaluated our approach by applying it to the recently released open-access ConSLAM dataset (Trzeciak et al., 2023b,a).

The ConSLAM dataset consists of four sequences of a construction site captured with a handheld system. It incorporates synchronized timestamped LiDAR scans, 9-axis IMU measurements, and red, green, and blue (RGB) and Near-infrared (NIR) camera images.

Given the TLS point cloud of sequence number two, we elaborate a half-centimeter-accurate BIM model.

We used the OA-LICalib library (Lv et al., 2020, 2022) to retrieve the extrinsic calibration parameters (rotation and translation) between the LiDAR and the IMU sensors.

5.2 Implementation details

While *Step 1* and *3* were implemented in Python, *Step 2* was written in C++.

5.2.1 Step 1: Reference session generation

In *Step 1*, to generate the reference session data ($\mathcal{S}_R$), the vertical Field of View (FoV) of the simulated LiDAR scans can be customized according to preferences. To achieve alignment with a TLS point cloud as a reference map, the simulated LiDAR scans encompass a range from -45 degrees to 45 degrees in the vertical FoV. However, in our experiments, while aligning the data with a BIM model, we observed improved ISC loop detection when no ceiling points were present in the simulated scans. Consequently, the scans are adjusted to cover only from 0 to -25 degrees in the vertical direction. In Blensor, during the LiDAR simulation process, the noise was set to a mean of zero with a standard deviation of 0.03 m, an angular resolution of 0.1728 degrees, and a maximum distance of 15 m.

5.2.2 Step 2: Query session generation, alignment, and correction

5.2.2.1 Step 2.1: Query session creation

In *Step 2*, to generate the query session from the real-word data ($\mathcal{S}_Q$), for the MDC step, we opted for using DLIO, because, in contrast to FAST-LIO2 (Xu et al., 2022), it does not require heavy downsampling of the point cloud for deskewing and registration. Hence, clean, undistorted scans with DLIO allow dense map reconstruction. As suggested by (Zhang et al., 2022), we reproduced the data in the *bagfiles* at a low rate (half of the original speed) to avoid errors during the distortion process. Regarding the key information saver, while it is possible to await a minimum variation on translation or rotation between consecutive scans, we opted to save scans given either a list of timestamps or after a specific interval of time has passed. This feature is convenient since we want to compare our results with existing ConSLAM GT poses. Therefore, we are mainly interested in specific frames with known timestamps. For the creation of ISCD we opted for $N_s = 60$, $N_r = 20$ (as suggested in (Kim et al., 2021)), $ISC_{\min} = 40$, and a maximum radius of 10 m.

5.2.2.2 Step 2.2: Inter-session loop detection with ISC

Nanoflann (Blanco and Rai, 2014) is used to create a KD-tree of 1D rotational invariant descriptors. A total of 100 (N_c) top candidates were chosen to evaluate in 2D after the 1D descriptor comparison; it is worth mentioning that the retrieval of correct correspondences is very sensitive to this value. A cosine similarity threshold $\epsilon = 0.3$ is used to filter out pairs of 2D descriptors that passed with the minimum distance among the possible column shifts k . Only column shifts of 10% of the total number of columns (i.e., 36 deg) are considered for the alignment. All YawGICP registrations in the ISC and KNN loops are done using parallel computations with OpenMP. Unlike conventional ICP implementations, when employing YawGICP, it is imperative to express the target point cloud (i.e., from the reference map) in the local coordinate system of the source scan (i.e., the point cloud to be aligned). Otherwise, the process will yield undesirable results. This shift is critical because the resulting transformation matrix is relative to the origin of the source scan, with the aim of rotating the point cloud from its local origin rather than the origin of the global coordinate system.

5.2.2.3 Step 2.2: KNN loops, pose-graph optimization and final ICP

The K-nearest neighbors used to create the submaps for KNN loop detection in the second step of optimization is 5. To ensure correct alignment with the BIM model as the reference map, we opted to omit the KNN loop detection process. This decision was made because this process tends to induce erroneous correspondences. Meanwhile, in Step 2.3.2 (Section 4.2.3), the pose-graph optimization is done with GTSAM using iSAM2; the following are the values of the variances of the different noise models: $\Sigma_L = \pi^2$ (significant noise for query session’s anchor node); $\Sigma_P = 1 \times 10^{-102}$ (prior noise for reference map poses and initial poses); $\Sigma_O = 1 \times 10^{-4}$ (for odometry constraints); $\Sigma_c = 0.5$ (robust noise for encounters, i.e., loop closure constraints). The parallel creation of spheres for target point cloud registration and the P2P ICP of the single source scans is done using OpenMP in C++. In our case, we use the following two maximum distances to calculate the fitness scores: $F_1 = 1$ cm and $F_2 = 3$ cm.

5.2.3 Step 3: Change detection and map update

In *Step 3*, the process is performed with Trimesh, OctoMap, and Open3D.

We use a P2P distance threshold of 0.3 m to calculate the positive and negative differences.

OGM2PGBM (Vega Torres et al., 2022), Scan Context (Kim and Kim, 2018), and LT-SLAM (Kim and Kim, 2022) were projects that we used as a reference and that are freely available online.

For the evaluation of the results, presented in the following section, the trajectories were compared against the ground truth using evo (Grupp, 2017) in TUM format (Sturm et al., 2012) and using the Umeyama alignment (Umeyama, 1991).

6 Results and analysis

In this section, we provide the results of our pipeline with respect to the alignment with an accurate TLS point cloud and with a BIM model as a reference map.

Table 1 shows the Absolute Pose Error (APE) summary of the different methods in each sequence of the ConSLAM dataset after alignment with the corresponding TLS point clouds. In the table, the performance of DLIO (with Umeyama alignment) is compared against the results of our framework after improving the DLIO trajectory with Indoor Scan Context (ISC) loop detection and after KNN loop detection and optimization. The results after the *final ICP* step correspond to the current ground truth (used to evaluate the methods); therefore, they are not numerical values for this step. Moreover, the results are compared against the original ConSLAM ground truth poses provided by the authors together with the dataset.

Furthermore, figures 9, 10, 11 illustrate the distribution of errors (translational and rotational) for each method in the various sequences. Figure 13, provides a visual representation of the resulted trajectories and 3D maps.

Method	S2 (225 m)		S3 (340 m)		S4 (275 m)		S5 (320 m)		Average	
DLIO	20.2	2.2	21.4	2.6	359.0	6.4	17.4	2.3	104.5	3.4
SC	20.1	2.2	24.3	2.6	358.6	6.4	18.7	2.3	105.4	3.4
KNN	9.0	1.4	34.6	4.3	53.4	3.5	11.2	2.5	27.1	2.9
ConSLAM	5.2	0.7	4.2	0.7	9.3	0.9	12.1	1.1	7.7	0.8

Table 1: Quantitative comparative results for each ConSLAM sequence (S2, S3, S4 and S5). Translational and angular APE RMSE in centimeters and degrees, respectively. Additionally, the length of each sequence is given in meters. SC refers to the results of DLIO after Scan Context loop detection and optimization, similarly KNN refers to the results after KNN loops. ConSLAM refers to the ground truth poses provided together with the dataset.

Notably, the errors exhibit an evident reduction across almost all sequences while the pipeline evolves.

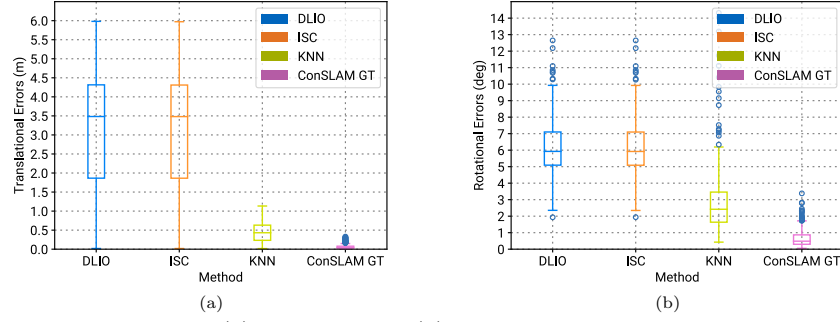


Fig. 9: Translational (a) and rotational (b) errors for sequence 4 after alignment with the respective TLS point cloud.

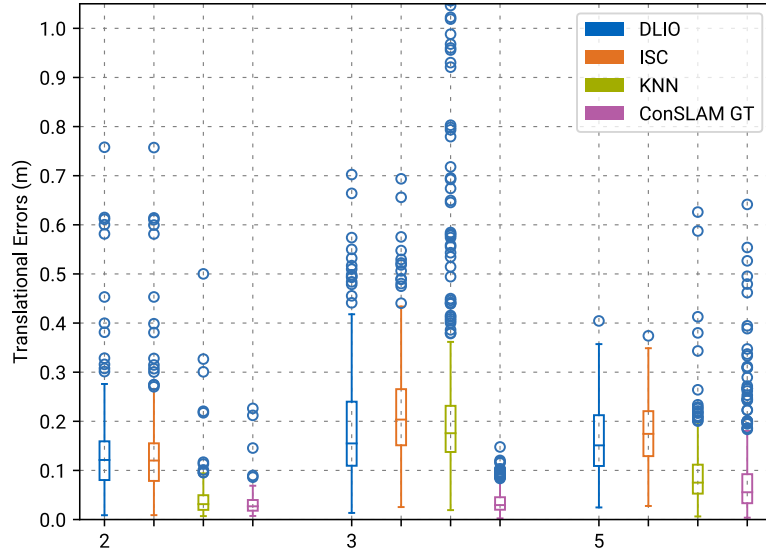


Fig. 10: Translational errors for sequences 2, 3, and 5 after alignment with the respective TLS point clouds.

The ISC loops primarily allow the critical first rough alignment between the query and reference sessions. Since only a few ISC loops are retained due to rigorous threshold criteria, the outcomes after ISC loop detection and optimization exhibit minimal

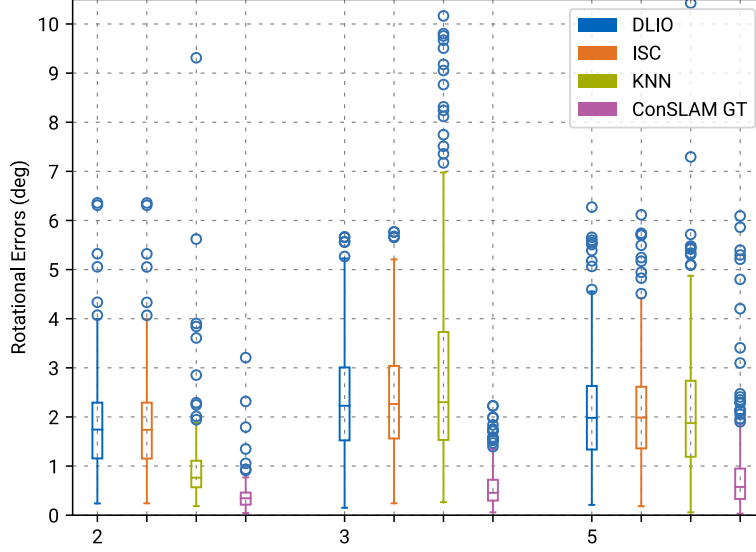


Fig. 11: Rotational errors for sequences 2, 3, and 5 after alignment with the respective TLS point clouds.

alteration in trajectory accuracy when compared to the initial results derived from DLIO.

On the other hand, the subsequent KNN loops exhibit a more pronounced impact on the results after ISC loops.

While the average rotational error, as depicted in Figure 11, experiences a significant decrease in sequences 2 and 4, it exhibits apparent stability or even an increase in sequences 3 and 5.

Regarding the GT poses provided with the ConSLAM dataset, although the RMSE for APE remains below 8 cm and 1 degree for translation and angular errors, respectively (as shown in the last column of Table 1), the maximum errors escalate to 20 or even 60 cm in sequences 2 and 5 (see Figure 10). While these significant discrepancies are in relatively small sections of the trajectories, it is also essential to recognize that for a LiDAR-based SLAM dataset, ground truth poses should ideally exhibit accuracy levels of at least one centimeter across the entire trajectory. This level of accuracy is now achievable in a highly automated manner with the proposed SLAM2REF framework.

Additionally, we demonstrate that it is possible to align and correct a 3D map using a BIM model as a reference map, despite significant deviations between the

current map and the reference BIM model (Scan-Map deviations of the types 1 and 2 as stated in the introduction, see Section 1). This significant level of deviation is particularly evident in the context of the ConSLAM construction site. Figure 12 and 13 depicts the results after alignment with BIM. Here, the error values after the final ICP step are visible since they do not coincide with the ground truth poses anymore.

Similar to the alignment process with the TLS point cloud presented previously, the error does not decrease after ISC loops; however, it notably reduces after the final ICP step. The translational RMSE of the APE decreases to 14.8 cm, while the rotational RMSE is 0.56 degrees.

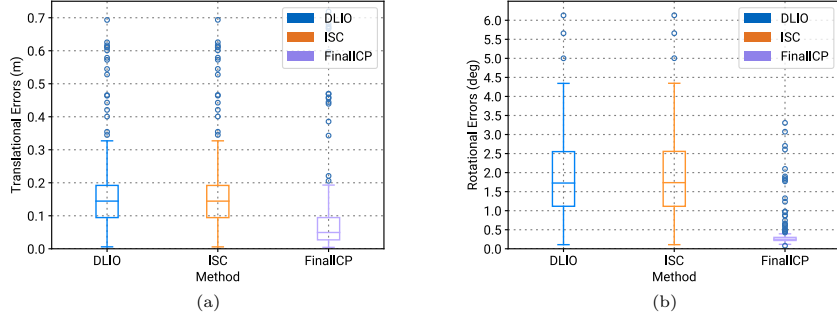


Fig. 12: Translational (a) and rotational (b) errors for the sequence 2 after alignment with BIM model.

Regarding the results of *Step 3*, the identified positive changes are highlighted in red within the final column of Figure 13. Furthermore, Figures 14a and 14b provide detailed visualizations of the discrepancies observed in sequence 2 following alignment with both the TLS point cloud and the BIM model, respectively. While the disparities with the TLS point cloud are relatively minor, involving slight shifts in the positions of certain fences and construction resources, the distinctions when compared to the BIM model (Figure 14b) are notably substantial. This serves to exhibit the robustness of our